Optiver

# Optiver Trading Challenge

Trading Challenge: Dual Listing trading algorithm

# Table of Contents

# 1 Introduction

In this trading challenge you will work on an algorithm to trade a dual listing of a stock. A “dual listing” is the listing of any security on two different exchanges. That means the same instrument is traded, in the same currency, but in two different places or exchanges, e.g. Paris and Milan.

The challenge introduces you to the concept of arbitrage, a theoretically risk-free trade, between different instruments. It demonstrates how these types of trading strategies, besides leading to a profit for the trader, help keep markets efficient and fairly priced. The specification is deceptively simple, but experience has shown that mastering this simple strategy touches on almost all of the understanding of order books, order types, and order matching methodologies, that more complex trading strategies require. It is highly recommended to attempt to obtain a full understanding of the ins and outs, including the technical aspects, of this strategy before moving on to more involved ones.

After finishing it, your algorithm will continuously provide liquidity in an illiquid listing of a share through inserting limit orders based on the prices of the liquid listing, ensuring to keep total outstanding positions hedged.

# 2 Trading Challenge

The exchange lists some stocks which have dual listings, named **<stock>** (the original) and **<stock>_DUAL** (the dual). For example, ASML and ASML_DUAL. Both stocks represent ownership in the same underlying company. Refer to the Instrument Reference in the Optibook Guide for more information.

If the underlying company is the same, it stands to reason that the prices to buy and sell the instruments at should also be the same. While that one-to-one price correspondence holds true in the long-run equilibrium, it does not always hold in the order books on the exchange. The order books are independent of each other, so we can have short-term deviations based on buying and selling pressures in the individual order books. A participant in one, might not be active in the other, and so on. The information is disjoint.

How does the information of disjoint trades and participants get synchronized from one order book to the other? This is where your algorithm steps in. For this challenge, the 'original' listings may be considered to be leading/more liquid; while the 'dual' listings should follow and mean-revert to their values. We can simply write this long-term relationship as $S = S_D$.

Implement the following two trading strategies based on the provided sample code in your devenv (a hint: make a copy of the sample algorithm before modifying it, for future reference).

## 2.1 Active Arbitrage Strategy

An active arbitrage trading strategy is one that checks for trading opportunities in a less liquid order book, based on the prices in a liquid order book, trading the opportunity with an IOC order, and then doing a hedge in the liquid instrument.

Think about the following: When are such trades theoretically profitable (consider details such as order types, prices, etc.)? What is the difference between an order, a trade and a position? What are all the hypothetical ways in which the order insertion could end, are there ways to "miss a trade"? How do you deal with those cases? How do the exchange position and outstanding order volume limits restrict the volumes of the orders you insert?

- Implement a profitable active trading strategy between a stock and its dual listing; ensuring to stay hedged as much as possible.

## 2.2 Passive / Quoting Strategy

By trading on the profitable opportunities, the market prices in the two order books become more in line with each other, but you effectively "take liquidity from the book" in doing so. Think about it this way, after your trade, there is less volume available to trade against for others, until it gets refilled. You've used up some of the book's capacity to allow trading.

Adding liquidity is the opposite action, where you thicken and/or tighten the order books by adding limit orders on both the bid and the ask side simultaneously and allow others to execute against these orders. This is also known as a market making strategy.

Think about the following: If you wish to show a price in the dual listing based on the prices in the original listing, which leads to a profit when it executes, what price should that be for the bid? And for the ask? What different scenarios could occur after you insert a limit order? When do your limit orders trade? What is that based on? How long do you keep limit orders in the market? How do you work with hedging in this strategy?

- Implement a profitable passive trading strategy in the same dual listing as in 2.1; ensuring to stay hedged as much as possible

### 2.3 Multiple Products / Abstraction

Depending on how you developed your code, the following might be either trivial or quite challenging. It is likely to require some abstraction into variables and functions. There are multiple dual listings available on the exchange, is it possible to trade both in a well-coded manner?

- Convert the code you implemented for the above strategies to run for both product-pairs simultaneously. Abstract your code, in the sense that it would be easy to swap in or out additional dual listings or to rename a pair. Are there any parameters you would like to set differently for the products?

## 3 Hints

Do not implement your whole algorithm at once. Start with a single piece of functionality and run it manually. Does it work as expected, what are the edge cases? Only when you are convinced you know exactly how to do each separate part, add it all together and run it on a loop to complete your algorithm.

Make extensive use of print()-statements in your algorithm to keep track of what it is doing. Don't assume, display all of the algorithms' state and decisions. It is exactly getting the details right which is complex about algorithm design, and as such, it is crucial to know what those details are. For added convenience in Jupyter you can use the *clear_output(wait=True)* command to clear a cell's output during runtime.

The workshops and your fellow students are there to help you out with all aspects of the challenge. Don't stay stuck on one part, but work through it by asking relevant questions, be that on coding, trading understanding, idea generation, and so on..

Good luck!